

Folien-Analyse — Webserver / HTTP / URL (Säule 4)

Quelle: webserver/Webserver-Day1 + HTTP-Folien aus html/Day3+Day4 (Kommunikation, Statuscodes, OSI). Gegen RFC/MDN verifiziert. Folienfehler mit . Themen-IDs S1–S7. Format: Q&A + Kontrast-Tabellen.

S1 — Aufgaben eines Webserver (Definition, hoch)

- **F: Was macht ein Webserver grundsätzlich?** A: HTTP-Anfragen entgegennehmen und beantworten — statische Ressourcen ausliefern oder Antwort dynamisch erzeugen/an Code delegieren.
- **F: Ablauf einer HTTP-Anfrage (Kette)?** A: auf Verbindung warten → Anfrage entgegennehmen (validieren, URL normalisieren) → interpretieren (Parameter auslesen, Antwort statisch/dynamisch erzeugen) → Antwort senden → Verbindung abbauen.
- **F: Weitere Aufgaben?** A: mehrere Anfragen parallel (Multithreading), Zuordnung per URI, Zugriffskontrolle, Caching, Komprimierung.

Lokal (file) vs. Webserver (http(s))

	Lokal (file)	Webserver (http/s)
Schema	file	http(s), ftp...
Rechte	des Browser-Users	des Server-Users, gefiltert durch Server-Config
Zugriff	nur lokal	von überall
HTML	nur statisch	dynamisch erzeugbar

Server-Erweiterung (dynamische Erzeugung)

Technik	Prinzip	Nachteil
CGI (RFC3875)	getrennter Prozess je Aufruf, URI/POST via stdin+Env, HTML via stdout	langsam (Prozess-Start), wenig Infrastruktur
SSI	Template-Tags in .shtml (#include,#echo,#if,#exec)	wenige Möglichkeiten, obsolet
Apache-Modul	eingebettet im Server-Prozess (mod_php...)	Code kann Server crashen
ISAPI	Microsoft IIS, DLL im Serverprozess	proprietär, plattformabhängig
gleiche Sprache (Tomcat/Node)	Server in selber Sprache wie Code	— (moderner Ansatz)
* Getrennte vs. eingebettete Prozesse: getrennt = stabiler/langsamer (CGI); eingebettet = schneller/riskanter (Modul/ISAPI).		

Fallen / Bugs

- **Falle:** CGI startet pro Request einen Prozess → langsam; eingebettete Module laufen im Serverprozess → schnell, aber Crash-Risiko.

Drill

- [Definition] Nenne zwei Aufgaben eines Webserver.

Lösung: HTTP-Anfragen beantworten; statische Ressourcen ausliefern bzw. dynamisch erzeugen.

- [MC] Welche Technik startet pro Aufruf einen eigenen Prozess?

Lösung: CGI.

S2 — HTTP-Methoden + Statuscodes (Definition/MC, hoch)

- **HTTP:** Protokoll zur Datenübertragung; Request/Response-Prinzip (keine Antwort ohne Anfrage); **zustandslos**; Request/Response = Header (Metadaten) + Body (Inhalt). HTTP/1.1 = RFC2068 (1997).
- **Parameterübergabe:** per URL (Query) ODER im Anfrage-Inhalt (Body).

Methoden (REST)

HTTP-Methoden geben an, welche Aktion auf einer Ressource ausgeführt werden soll:

Methode	Zweck
GET	Ressource abrufen (Parameter in URL)
POST	Daten senden/anlegen (im Body)
PUT	Ressource ersetzen/anlegen
DELETE	Ressource löschen

Statuscode-Klassen

Der Statuscode einer Antwort ist dreistellig; die erste Ziffer ordnet ihn einer Klasse zu:

Klasse	Bedeutung	Beispiele
1xx	Information	100 Continue
2xx	Erfolg	200 OK, 201 Created, 204 No Content
3xx	Umleitung	301 Moved Permanently, 302 Found, 304 Not Modified, 307 Temporary Redirect
4xx	Client-Fehler	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server-Fehler	500 Internal Server Error, 503 Service Unavailable

FOLIE (html/Day4): kryptische Klassen-Beschreibungen ("Antwort im ersten Stock", "verhindert Zeitüberschreitung") = Vortragsnotizen/OCR-Müll. Korrekte Bedeutungen siehe Tabelle (MDN).

OSI-Einordnung (7 Schichten, Auszug)

Das OSI-Modell schichtet Netzwerkkommunikation; HTTP liegt ganz oben auf der Anwendungsschicht:

- 7 Anwendung: HTTP(S), DNS, DHCP, FTP · 4 Transport: TCP/UDP · 3 Vermittlung: IP/Router · 2 Sicherung: Ethernet/MAC · 1 Bitübertragung. HTTP liegt auf Schicht 7, TCP auf 4.

Fallen / Bugs

- **MC-Falle:** 401 = nicht authentifiziert (Login fehlt) vs. 403 = authentifiziert, aber verboten.
- **MC-Falle:** 301 (permanent) vs. 302/307 (temporär).

Drill

- [MC] Statuscode-Klasse für "Seite nicht gefunden"?

Lösung: 4xx (404).

- [MC] 401 vs 403 — Unterschied?

Lösung: 401 = Authentifizierung fehlt; 403 = trotz Auth verboten.

S3 — Redirect vs. Forward (Definition/MC, hoch)

	Redirect	Forward
Wer leitet	Client (Browser)	Server (intern)
Mechanismus	3xx + Location-Header → Browser stellt 2. Request	Server reicht Request intern weiter

	Redirect	Forward
URL im Browser	ändert sich	bleibt gleich
Roundtrips	2	1
Express	<code>res.redirect(path)</code>	<code>res.render(view)</code>

Fallen / Bugs

- **Kontrast:** Redirect macht zweiten Request (URL wechselt); Forward ist serverintern (URL bleibt).

Drill

- [MC] URL im Browser ändert sich — Redirect oder Forward?

Lösung: Redirect.

S4 — Cookies (Definition, hoch)

- **F: Warum Cookies?** HTTP ist zustandslos → Cookies tragen Zustand (Session, Login, Warenkorb).
- **F: Ablauf?** Server sendet `Set-Cookie`-Header → Browser speichert → hängt bei jedem passenden Request `Cookie:`-Header automatisch an.
- **Anatomie:** `key=value` (+Attribute mit `;`). Nur `key=value` geht zurück.

Attribut	Funktion
Domain/Path	Geltungsbereich
Expires/Max-Age	Lebensdauer; ohne → Session-Cookie
Secure	nur über HTTPS
HttpOnly	kein JS-Zugriff (<code>document.cookie</code>)

Drill

- [Definition] Wie kommt ein Cookie zum Browser?

Lösung: Server-Header `Set-Cookie`, Browser speichert und sendet künftig im `Cookie`-Header zurück.

S5 — URLs absolut vs. relativ (MC, hoch)

`<schema>://<server>:<port>/<pfad>?<query>#<fragment>`

Form	Bedeutung	Beispiel
absolut	Schema+Host+Pfad	<code>https://x.de/a/b</code>
<code>//host/...</code>	protokoll-relativ (Schema übernommen)	<code>//cdn.x.de/a</code>
<code>/pfad</code>	root-relativ (gleicher Server)	<code>/img/a.png</code>
<code>.././,./,datei.html</code>	dokument-relativ	<code>../b.html</code>
<code>#id</code>	gleiche Ressource, anderes Element	<code>#kap2</code>

- Standard-Ports: http=80, https=443. Query: `name=wert` mit `&`. Fragment `#...` geht NIE zum Server.

Fallen / Bugs

- **MC-Falle:** `/start` ist root-**relativ**, nicht absolut.
- **Vorteil relativ:** portabel bei Domain-/Pfadwechsel.

Drill

- [MC] absolut/relativ: `https://x.de`, `/a`, `../b`, `//cdn/a`?

Lösung: absolut, root-relativ, dokument-relativ, protokoll-relativ.

S6 — Sessions vs. Cookies (Definition/MC, mittel)

	Cookie	Session
Speicherort	Client (Browser)	Server (Daten), Client hält nur Session-ID
Inhalt	key=value-Strings	beliebige serverseitige Daten
Sicherheit	clientseitig manipulierbar	sicherer (Daten am Server)
Mechanik	Set-Cookie/Cookie	meist über Cookie mit Session-ID implementiert

Fallen / Bugs

- **Kontrast:** Session-Daten liegen am Server; das Cookie transportiert nur die ID. Sessions müssen (in Node) selbst implementiert werden.

Drill

- [MC] Wo liegen Session-Daten typischerweise?

Lösung: auf dem Server; der Client hält nur die Session-ID (im Cookie).

S7 — Request/Response-Aufbau (Definition, mittel)

Request

Eine HTTP-Anfrage besteht aus Startzeile, Headern und optionalem Body:

- Zeile 1: **Methode + URL + Protokollversion** (`GET /a HTTP/1.1`).
- Header (z.B. `Host`, `User-Agent`), Format `Headername: Wert`.
- Leerzeile, dann optional Body.

Response

Die Server-Antwort ist ähnlich aufgebaut, beginnt aber mit einer Statuszeile:

- Zeile 1: **Statuszeile** (`HTTP/1.1 200 OK`).
- Header (`Content-Type` ...).
- Leerzeile (Headerabschluss), dann Body.

Fallen / Bugs

- **Falle:** Leerzeile trennt Header von Body — sowohl in Request als auch Response.

Drill

- [Definition] Woraus besteht eine HTTP-Response?

Lösung: Statuszeile (Protokoll, Code, Text), Header, Leerzeile, Body.